What is Dynamic Rendering?

With dynamic rendering, content is rendered on the server for each user at **request time** (when the user visits the page). There are a couple of benefits of dynamic rendering:

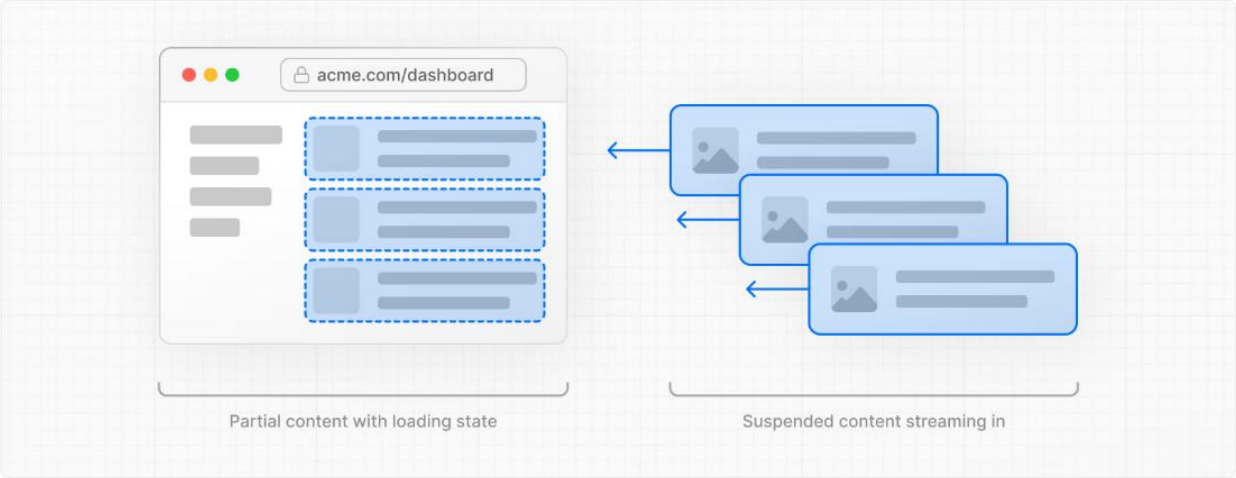
- **Real-Time Data** - Dynamic rendering allows your application to display real-time or frequently updated data. This is ideal for applications where data changes often.
- **User-Specific Content** - It's easier to serve personalized content, such as dashboards or user profiles, and update the data based on user interaction.
- **Request Time Information** - Dynamic rendering allows you to access information that can only be known at request time, such as cookies or the URL search parameters.

==========

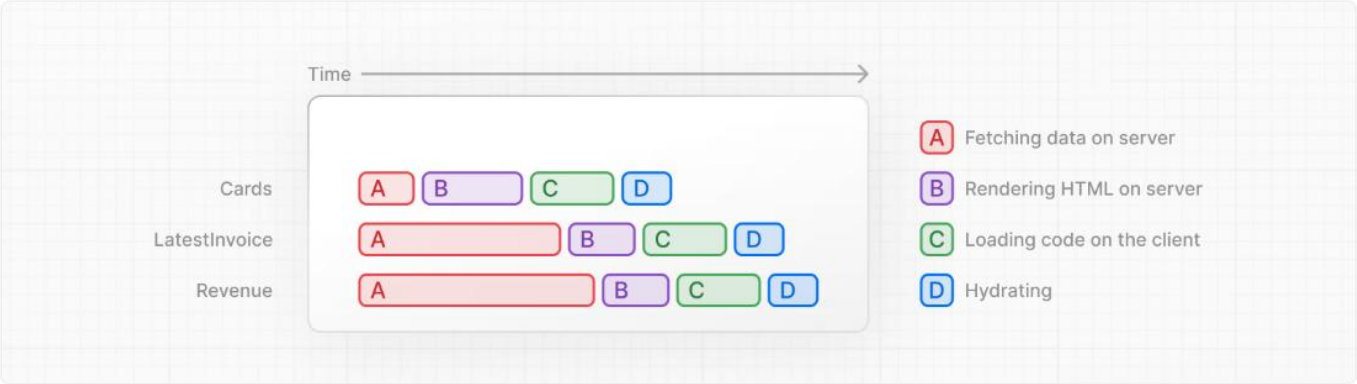
With dynamic rendering, **your application is only as fast as your slowest data fetch**.

==========

Streaming is **a data transfer technique** that allows you to break down a route into smaller "**chunks**" and **progressively stream them** from the server to the client as they become ready.



==========



Streaming works well with React's component model, as each component can be considered a *chunk*.

==========

There are two ways you implement streaming in **Next.js**:

1. At the page level, with the **loading.tsx** file (which creates <**Suspense**> for you).
2. At the component level, with <**Suspense**> for more granular control.

==========

What is one advantage of streaming?

C

Chunks are rendered in parallel, reducing the overall load time

✓ Correct

One advantage of this approach is that you can significantly reduce your page's overall loading time.

==========

A few things are happening here:

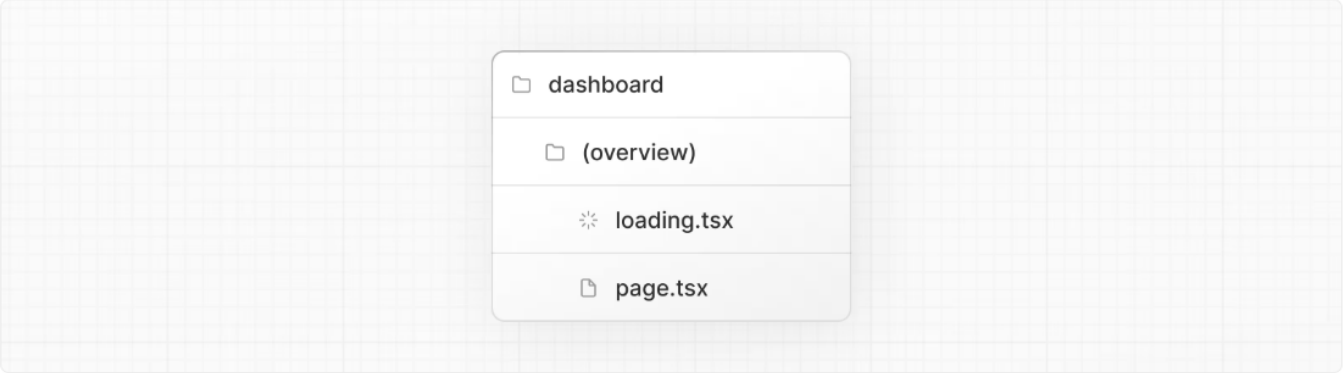
1. `loading.tsx` is a special Next.js file built on top of React Suspense. It allows you to create fallback UI to show as a replacement while page content loads.
2. Since `<SideNav>` is static, it's shown immediately. The user can interact with `<SideNav>` while the dynamic content is loading.
3. The user doesn't have to wait for the page to finish loading before navigating away (this is called interruptable navigation).

```tsx
export default function Loading() {
    return <div>Loading...</div>;
}
```

==========

Since `loading.tsx` is a level higher than `/invoices/page.tsx` and `/customers/page.tsx` in the file system, it's also applied to those pages.

We can change this with Route Groups ↗. Create a new folder called `/(overview)` inside the dashboard folder. Then, move your `loading.tsx` and `page.tsx` files inside the folder:



Now, the `loading.tsx` file will only apply to your dashboard overview page.

Route groups allow you to organize files into logical groups without affecting the URL path structure. When you create a new folder using parentheses `()`, the name won't be included in the URL path. So `/dashboard/(overview)/page.tsx` becomes `/dashboard`.

==========

Here, you're using a route group to ensure loading.tsx only applies to your dashboard overview page. However, you can also use route groups to separate your application into sections (e.g. (marketing) routes and (shop) routes) or by teams for larger applications.

==========

Suspense **allows you to defer rendering parts of your application** until some condition is met (e.g. data is loaded). You can wrap your dynamic components in Suspense. Then, pass it a fallback component to show while the dynamic component loads.

==========

Deciding where to place your Suspense boundaries

Where you place your Suspense boundaries will depend on a few things:

1. How you want the user to experience the page as it streams.
2. What content you want to prioritize.
3. If the components rely on data fetching.

Take a look at your dashboard page, is there anything you would've done differently?

Don't worry. There isn't a right answer.

- You could stream the **whole page** like we did with `loading.tsx` ... but that may lead to a longer loading time if one of the components has a slow data fetch.
- You could stream **every component** individually... but that may lead to UI *popping* into the screen as it becomes ready.
- You could also create a *staggered* effect by streaming **page sections**. But you'll need to create wrapper components.

Where you place your suspense boundaries will vary depending on your application. In general, it's good practice to move your data fetches down to the components that need it, and then wrap those components in Suspense. But there is nothing wrong with streaming the sections or the whole page if that's what your application needs.

Don't be afraid to experiment with Suspense and see what works best, it's a powerful API that can help you create more delightful user experiences.

==========

In general, what is considered good practice when working with Suspense and data fetching?

C

Move data fetches down to the components that need it

✓ Correct

By moving data fetching down to the components that need it, you can create more granular Suspense boundaries. This allows you to stream specific components and prevent the UI from blocking.

==========

URLSearchParams is a Web API that provides utility methods for manipulating the URL query parameters.

Instead of creating a complex string literal, you can use it to get the params string like ?page=1&query=a.

Note: The URL is updated without reloading the page, thanks to Next.js's client-side navigation.

```
import { usePathname, useRouter, useSearchParams } from 'next/navigation';
  const searchParams = useSearchParams()
  function handleSearch(term: string) {
    const params = new URLSearchParams(searchParams)
    const pathname = usePathname()
    const { replace } = useRouter()
    if(term) {
      params.set('query', term)
    } else {
      params.delete('query')
    }
    console.log(term)
    replace(`${pathname}?${params.toString()}`)
  }
```

==========

`defaultValue` vs. `value` / Controlled vs. Uncontrolled

If you're using state to manage the value of an input, you'd use the `value` attribute to make it a controlled component. This means React would manage the input's state.

However, since you're not using state, you can use `defaultValue`. This means the native input will manage its own state. This is okay since you're saving the search query to the URL instead of state.

==========

```tsx
export default async function Page(props: {
    searchParams?: Promise<{
        query?: string;
        page?: string;
    }>;
}) {

    const searchParams = await props.searchParams
    const query = searchParams?.query || ''
    const currentPage = parseInt(searchParams?.page || '1') || 1

    return <div className="w-full">
        <div className="flex w-full items-center justify-between">
            <h1 className={`${lusitana.className} text-2xl`}>Invoices</h1>
        </div>

        <div className="mt-4 flex items-center justify-between gap-2 md:mt-8">
            <Search placeholder="Search Invoices..." />
            <CreateInvoice />
        </div>

        <Suspense key={query + currentPage} fallback={<InvoicesTableSkeleton />}>
            <Table query={query} currentPage={currentPage} />
        </Suspense>

        <div className="mt-5 flex w-full justify-center">
            {/* <Pagination totalPages={totalPages} /> */}
        </div>
    </div>
}
```

==========

When to use the `useSearchParams()` hook vs. the `searchParams` prop?

You might have noticed you used two different ways to extract search params. Whether you use one or the other depends on whether you're working on the client or the server.

- `<Search>` is a Client Component, so you used the `useSearchParams()` hook to access the params from the client.
- `<Table>` is a Server Component that fetches its own data, so you can pass the `searchParams` prop from the page to the component.

As a general rule, if you want to read the params from the client, use the `useSearchParams()` hook as this avoids having to go back to the server.

==========

React Server Actions allow you **to run asynchronous code** directly on the server. They eliminate the need to create API endpoints to mutate your data. Instead, you write asynchronous functions that execute on the server and can be invoked from your **Client or Server Components**.

Security is a top priority for web applications, as they can be vulnerable to various threats. This is where Server Actions come in. They include features like **encrypted closures**, **strict input checks**, **error message hashing**, **host restrictions**, **and more** — all working together to significantly enhance your application security.

==========

In React, you can use the action attribute in the **<form> element to invoke actions**. The action will automatically receive the native FormData object, containing the captured data.

For example:

```
// Server Component
export default function Page() {
  // Action
  async function create(formData: FormData) {
    'use server';

    // Logic to mutate data...
  }

  // Invoke the action using the "action" attribute
  return <form action={create}>...</form>;
}
```

==========

An advantage of invoking a Server Action within a Server Component **is progressive enhancement** - forms work even if JavaScript has not yet loaded on the client. For example, without slower internet connections.

==========

Next.js with Server Actions

Server Actions are also deeply integrated with Next.js caching. When a form is submitted **through a Server Action**, not only can you use the action to mutate data, but **you can also revalidate the associated cache** using APIs like **revalidatePath** and **revalidateTag**.

==========

What's one benefit of using a Server Actions?

B

Progressive Enhancement.

✓ Correct

That's right! This allows users to interact with the form and submit data even if the JavaScript for the form hasn't been loaded yet or if it fails to load.

==========

By adding the 'use server', you mark all the exported functions within the file as **Server Actions**. These server functions can then be imported and **used in Client and Server components**. Any functions included in this file that are *not* used **will be automatically removed from the final application bundle**.

==========

By adding the **'use server'**, you mark all the exported functions within the file as **Server Actions**. These server functions can then be imported and used in **Client and Server components**. Any functions included in this file that are *not* used will be automatically removed from the final application bundle.

You can also write Server Actions directly inside Server Components by adding "**use server**" inside the action.

==========

Good to know: In HTML, you'd pass a URL to the `action` attribute. This URL would be the destination where your form data should be submitted (usually an API endpoint).

However, in React, the `action` attribute is considered a special prop - meaning React builds on top of it to allow actions to be invoked.

Behind the scenes, Server Actions create a `POST` API endpoint. This is why you don't need to create API endpoints manually when using Server Actions.

==========

Tip: If you're working with forms that have many fields, you may want to consider using the `entries()` method with JavaScript's `Object.fromEntries()`.

==========

Type validation and coercion

It's important to validate that the data from your form aligns with the expected types in your database. For instance, if you add a `console.log` inside your action:

```
console.log(typeof rawFormData.amount);
```

You'll notice that `amount` is of type `string` and not `number`. This is because `input` elements with `type="number"` actually return a string, not a number!

==========

To solve the above issue:

```
const FormSchema = z.object({
    id: z.string(),
    customerId: z.string(),
    amount: z.coerce.number(),
    status: z.enum(['pending', 'paid']),
    date: z.string(),
})

const CreateInvoice = FormSchema.omit({ id: true, date: true })
```

==========

Since you're updating the data displayed in the invoices route, you want to clear this cache and trigger a new request to the server. You can do this with the `revalidatePath` function from Next.js.

Once the database has been updated, the /dashboard/invoices path will be revalidated, and fresh data will be fetched from the server.

==========